



YOUR TECHNOLOGY & INNOVATION PARTNER

Modern Agents Framework Overview

Let make sense first



- MAF stand for **Modern Agents Framework**
- What is MAF?
 - Imagine:
 - You don't talk with AI once.
 - You work with system can think, brainstorm,...
 - It can use tools , workflows, APIs,...
 - It can know what to do next.
 - MAF = the system that **runs** AI agents, not just prompts

Table of contents



1. Architecture to Framework
2. Core Capabilities of MAF
3. AF Design Patterns
4. MAF Landscape
5. Work with LangChain Framework
6. Use cases
7. Q&A

1. ARCHITECTURE TO FRAMEWORK

Architecture to Framework

Recap

Agents ≠ LLMs

Agents make decisions, do **ACT**.

5 components to build agent

Brain -> Sense -> Hand -> Mind -> Guardrails.

Ways to assemble

- Chain, Agentic Loop, DAGs, MIX (Chain with agentic-loop)

Start from the work, not the model

Understand what the task *requires* before choosing tools.

Humans stay in the loop

Safety, approvals, and accountability are architectural decisions.

Why MAF exist?



“If you already know how to build an agent, why do you need a framework at all?”

- An agent framework is **NOT** a wrapper around an LLM. It is an **execution system for decision-making entities**.

Sample: [Demo code](#)

2. CORE CAPABILITIES OF MAF

Core Capabilities of MAF

“What capabilities does this framework provide?”

State

Memory

Planning

Orchestration

Streaming

Human_in_loop

Problem to solve

“Build an AI agent that helps a hospital schedule a patient’s follow-up appointment after a medical consultation.”

Human action

Receive information
from doctor



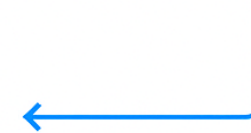
Access
System



Verify
information



Nurse
Confirmation



Add-in
notification

State

“If the agent forgot this, would it still behave correctly?”

Memory

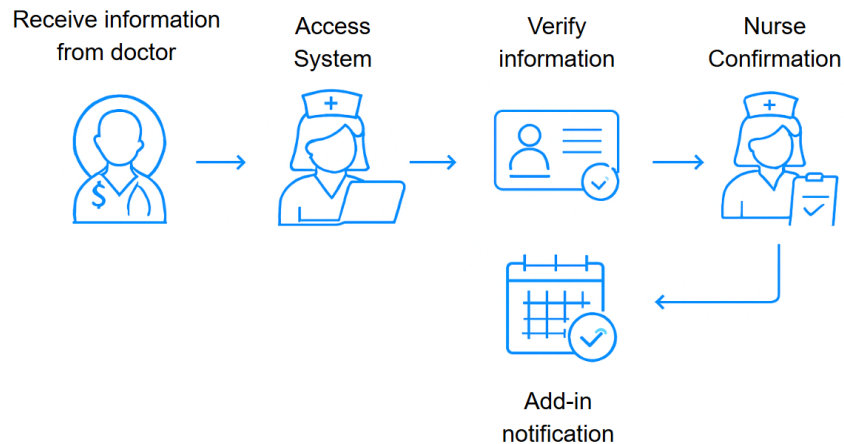
“Memory is a design choice, not a feature toggle.”

Planning

“Real agents must adapt, not just execute.”

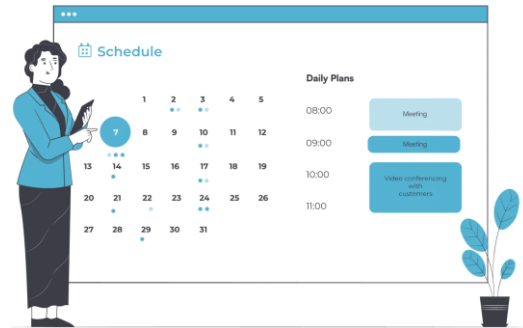
Orchestration

Applied tools - “Tools are power — power requires control.”



MAF **requirements** for this capabilities.

“How do humans see and control what the agent is doing?”



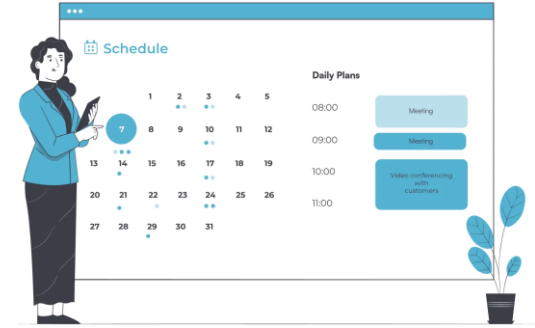
Streaming

Human-in-loop

Progress updates	Approval decision
Read-only	Can change outcome
Build trust	Build safety

MAF **requirements** for this capabilities.

“How do humans see and control what the agent is doing?”



Problem to solve

“Build an AI agent that helps a hospital schedule a patient’s follow-up appointment after a medical consultation.”

Agent State:

Current step: Waiting for approval, Status:
In progress

Planning:

1. Check doctor availability
2. Propose appointment time
3. Request human approval
4. Confirm appointment

Agent Memory:

Patient ID, Assigned doctor, Preferred
time, Rejected time

Orchestrating:

- Hospital patient system
- Doctor calendar
- Notification service

Human-in-loop:

- Doctor: Approve / Reject / Modify

Streaming:

- Hospital patient system
- Doctor calendar
- Notification service

3. AF DESIGN PATTERNS

AF Design Patterns

- The capabilities tell us what must exist, design patterns tell us how these pieces are arranged.
- **3 STEPS** for choosing design pattern:
 - **Define** your requirements.
 - **Review** the common agent design pattern.
 - **Select** design pattern based on workload.

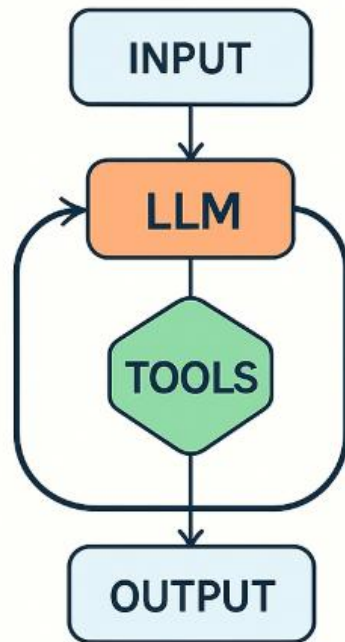
#1 Single-Agent

Characteristic:

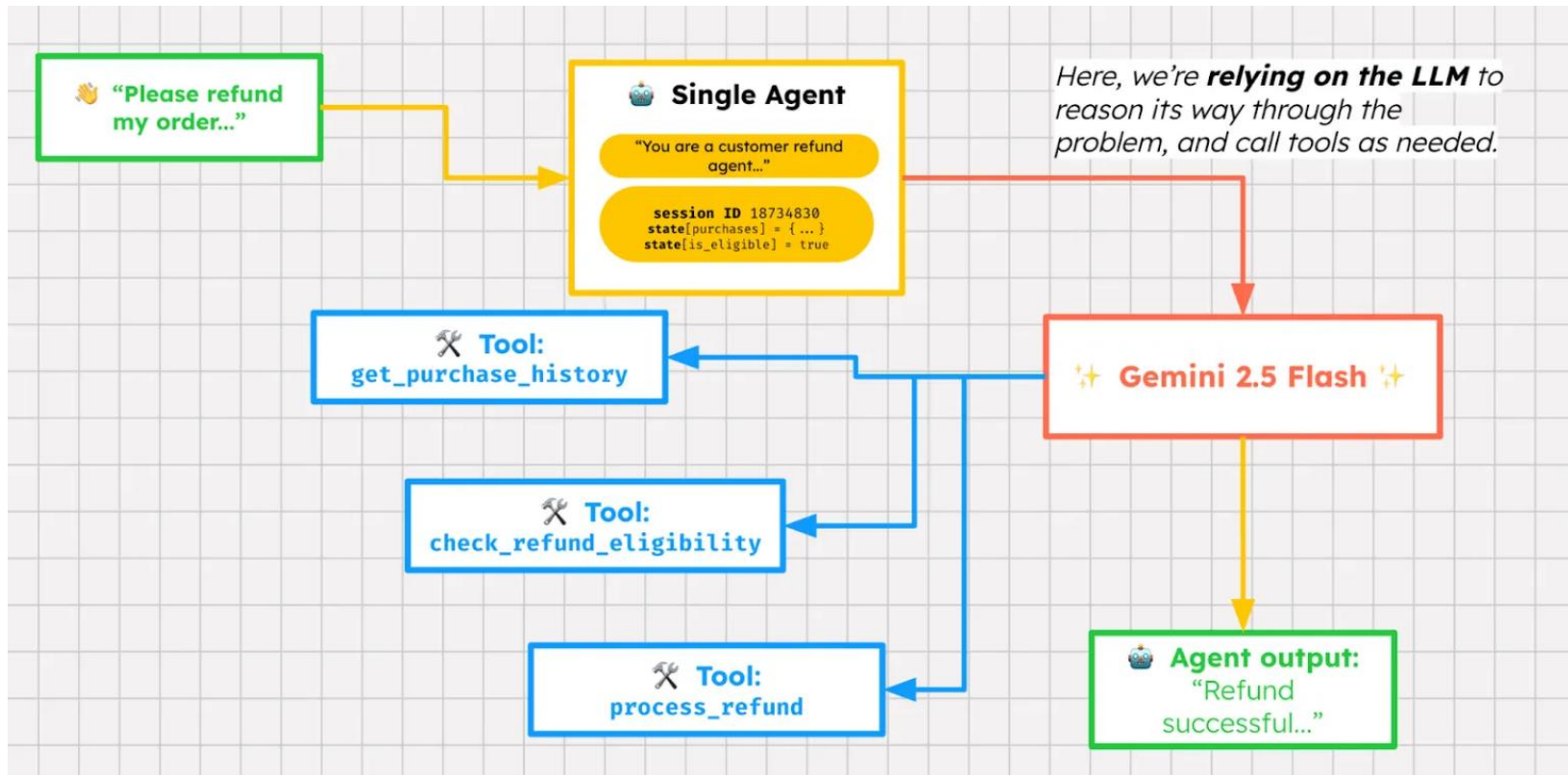
- Simple execution loop
- One decision-maker
- Tight control

“Everything happens inside **one decision loop**: the same agent **thinks, decides and executes.**”

Single-Agent



#1 Single-Agent



#2

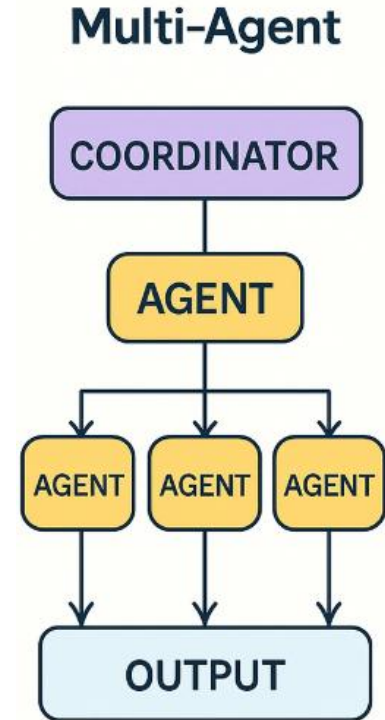
Multi-Agent

Characteristic:

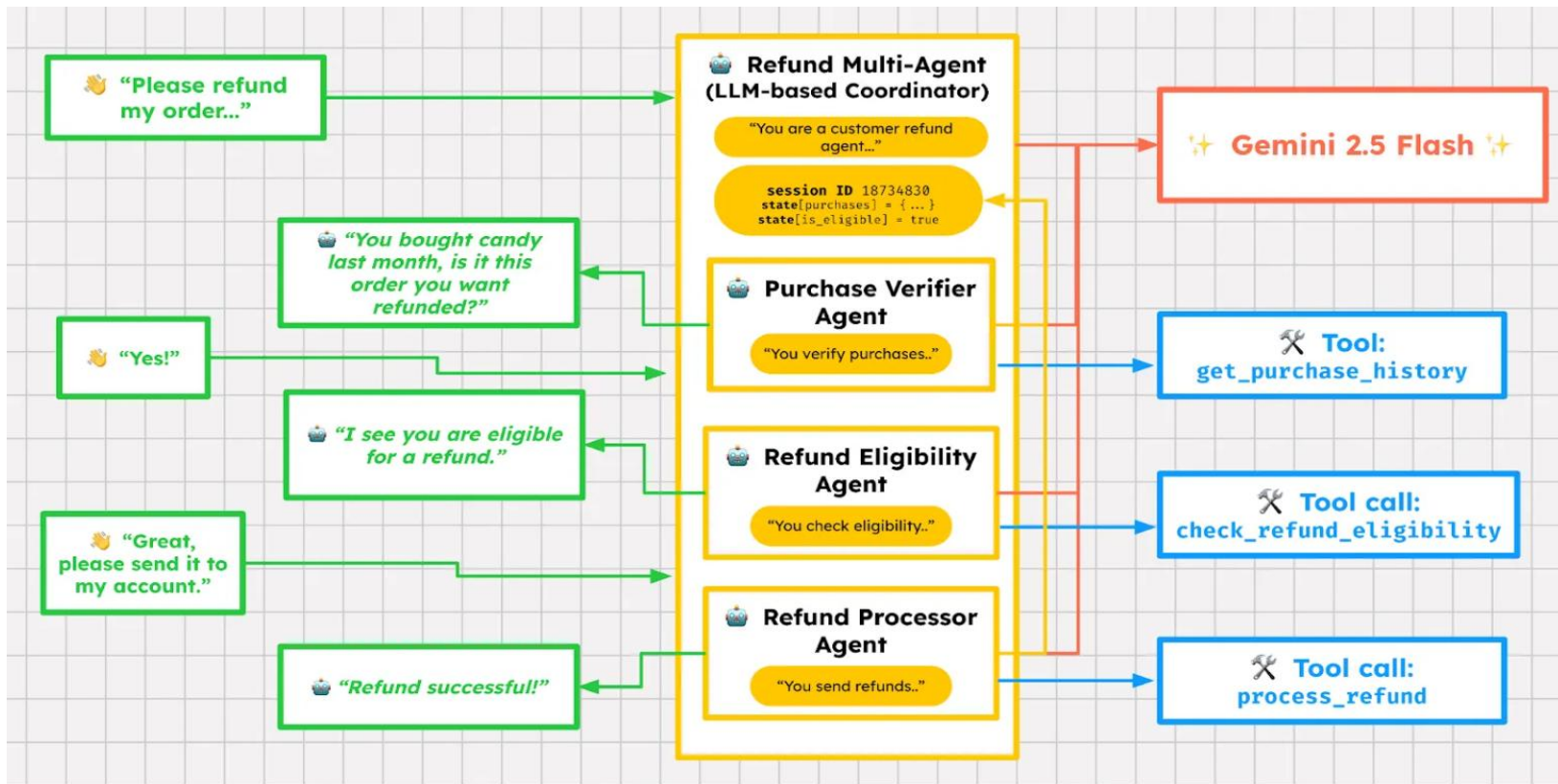
- Separation of concerns
- Agents communicate with agents
- Roles are explicit

“These agents work in parallel or in sequence, and their results are **combined into a final output.**”

Multi-agent = multiple brains, coordinated decisions.



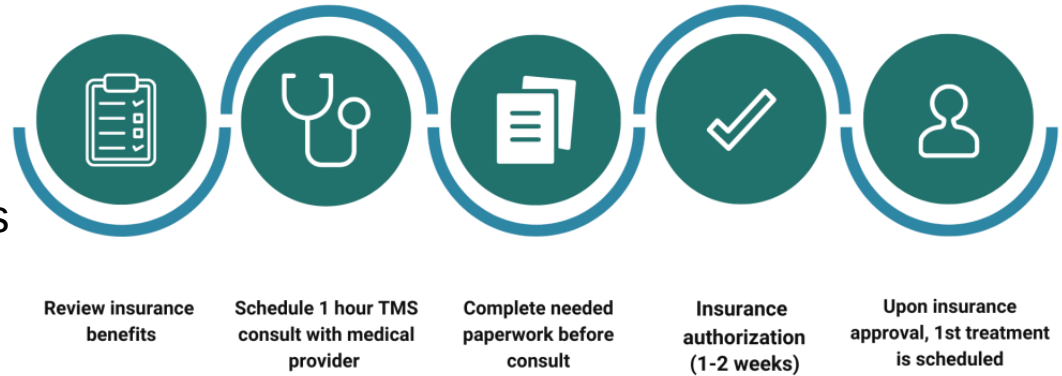
#2 Multi-Agent



#3 Orchestrated workflow

Characteristic:

- Deterministic flow
- Explicit states and transitions
- Easy to audit and debug

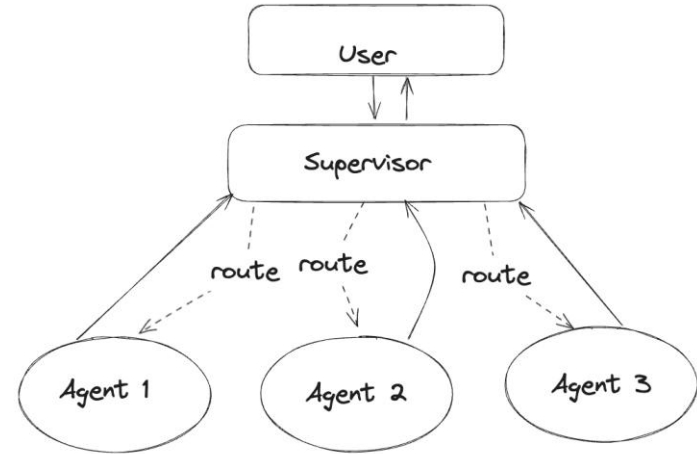


- Agents as **nodes** in a controlled flow
- Knowledge based:
 - DAGs (Directed Acyclic Graphs)
 - State machines
 - Event-Driven Agents

#4 Supervisor/Manager

Characteristic:

- Delegation and oversight
 - Central decision authority
 - Conflict resolution
-
- Frameworks are opinions about **how agents should coordinate decision-making**.
 - Its role is to **understand the user's goal**, then **delegate tasks** to the most appropriate agents.



Supervisor pattern = central control with distributed execution.

Let's take a break

10 - 15 minutes

Waiting for...

 LangChain

 LangGraph

crewai



Semantic Kernel

 AutoGen

4. MAF LANDSCAPE

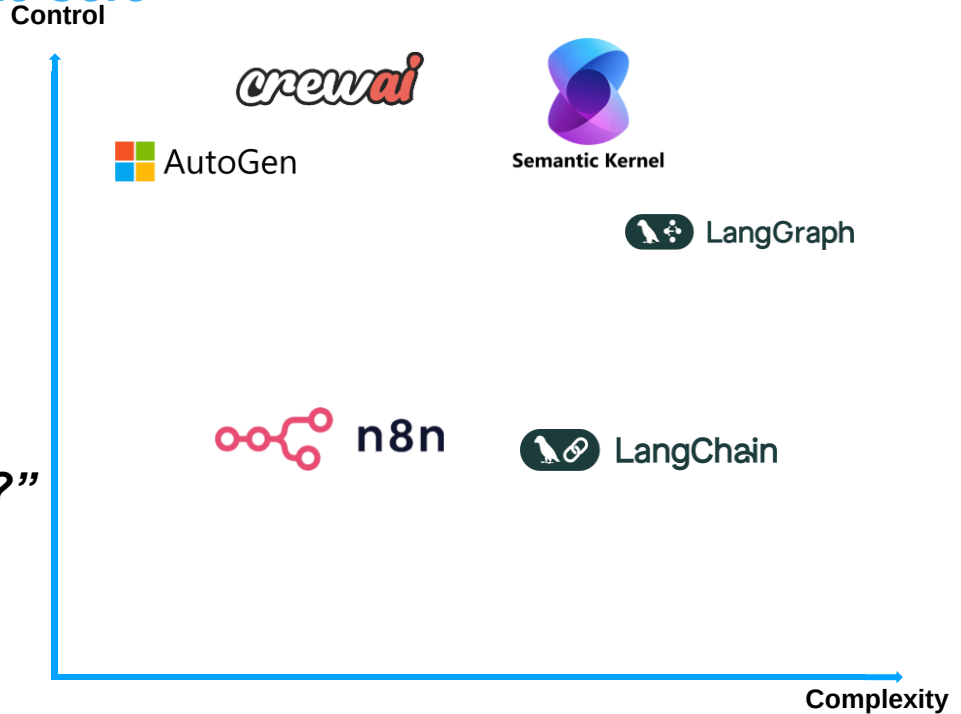
MAF Landscape

“Why frameworks look different?”

“Why frameworks look different?”

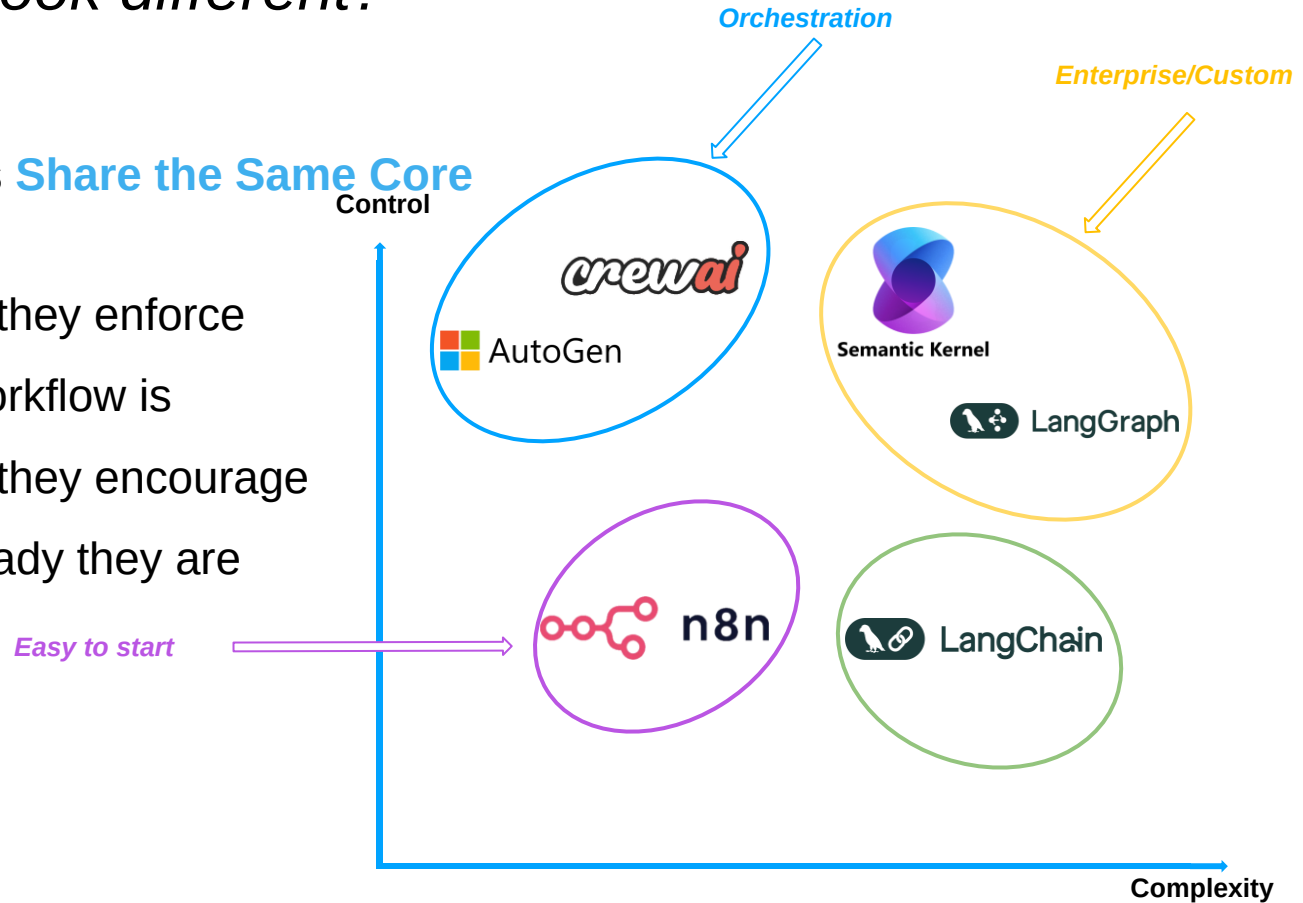
- All Modern Frameworks **Share the Same Core**
- **Differ in:**
 - How much control they enforce
 - How explicit the workflow is
 - How many agents they encourage
 - How production-ready they are

“For my project, which one should I use?”



“Why frameworks look different?”

- All Modern Frameworks **Share the Same Core**
- **Differ in:**
 - How much control they enforce
 - How explicit the workflow is
 - How many agents they encourage
 - How production-ready they are



Why Agent Frameworks Have Trade-offs

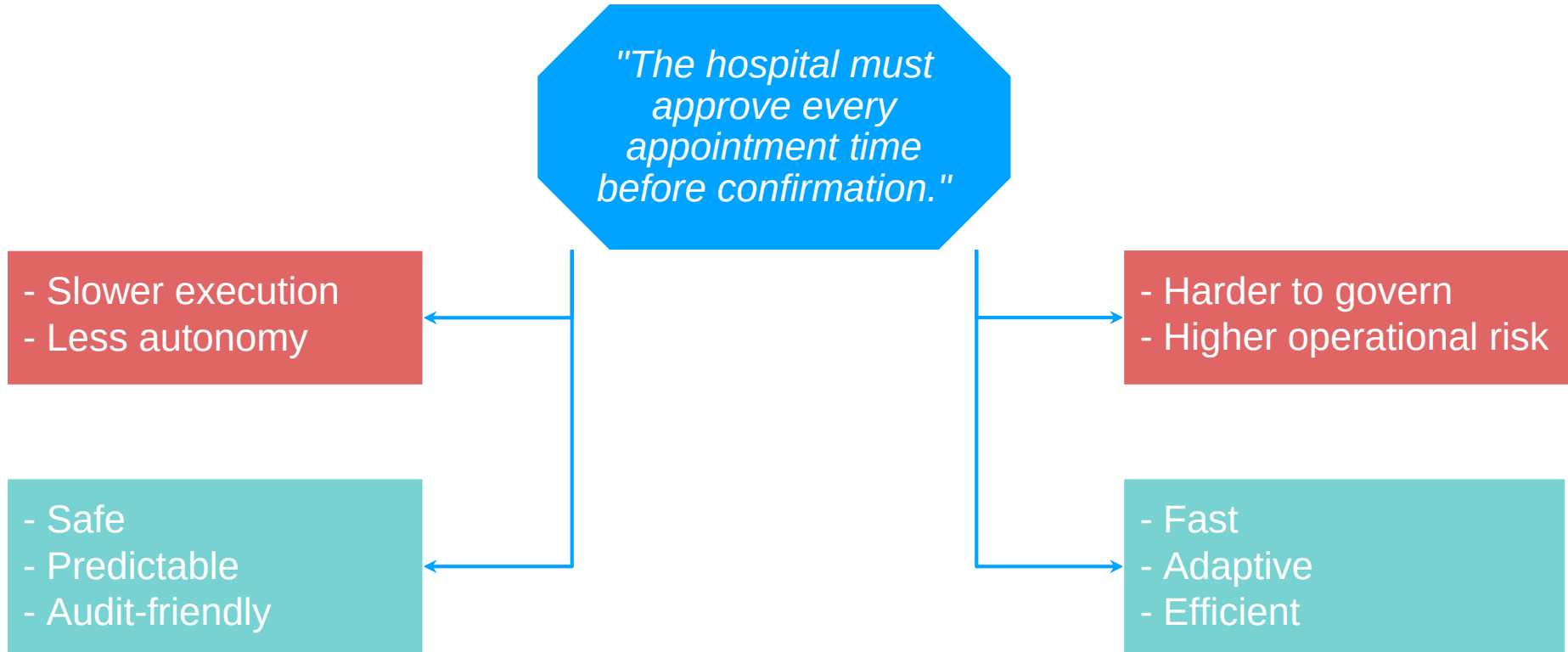
- No framework can optimize everything at once
- Improving one dimension often weakens another
- Frameworks encode **design priorities**, not perfection

Key Dimensions comparing trade-offs

- Control vs Flexibility
- Simplicity vs Power
- Predictability vs Emergence
- Debuggability vs Creativity
- Speed vs Safety

“Framework choice is risk management, not feature selection”

Control and Flexibility



"Control protects you from risk. Flexibility gives the agent freedom."

Talking: Start build for Hospital agent

- You want this agent to be creative (**Flexibility**) or absolutely comply with regulations (**Control**)?

Choice A	Choice B	The Strategic Reality
Safety	Speed	Safety (Hospital) requires human-in-the-loop or guardrails, which inevitably results in Slower Execution .
Control	Flexibility	Control protects the brand from operational risk. Flexibility allows the agent to be adaptive and efficient.
Debuggability	Creativity	Structured flows (Graph-style) are easier to fix, whereas Creative agents (Autonomous) are harder to govern.
Predictability	Emergence	Predictability is audit-friendly (Railway). Emergence is intelligent but can lead to "surprises".

Who are you hiring?

- Do you need to develop a specialized agent?

Q1: Does it need to follow a strict procedure (e.g., Bank)?

- **YES:** You need a "**Railway System.**" Choose **LangGraph.**
- **NO:** Choose **n8n**

Q2: Do you need multiple Agents to discuss with each other?

- **YES:** You need a "**Software Squad.**" Choose **AutoGen** or **CrewAI.**
- **NO:** choose **LangChain**

Q3: Do you just need to quickly make a simple question-answering application?

- **YES:** You need a "**Straight Sidewalk.**" Choose **LangChain.**
- **NO:** choose **LangGraph**

Chain-style



“A straight sidewalk. Easy to walk. No surprises.”

- Strength Implicit:

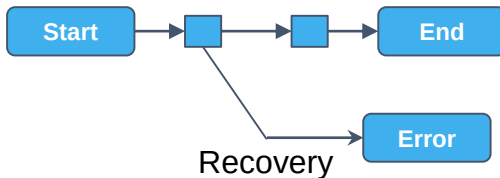
- Clear, simple, obvious path

- Trade-offs implicit:

- Can't handle detours



Workflow Framework



“A railway system. Controlled tracks. You must follow the path.”

- Strength Implicit:

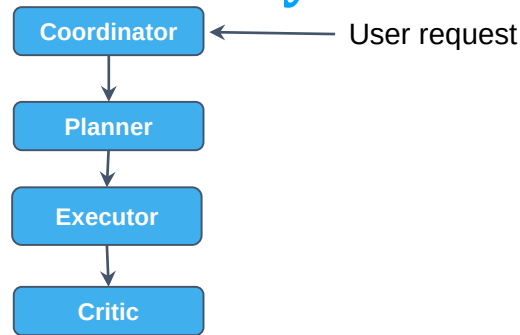
- Predictable, safe

- Trade-offs implicit:

- Must design tracks first



Multi-agent style



- Multi-agent isn't chaos.
- Shows **roles** clearly
- Still represents multi-agent collaboration



Summarize with example

Scenario	Priority	Winning Framework
Banking Transfer	Safety & Predictability	LangGraph (The Railway)
Creative Content Blog	Flexibility & Speed	LangChain (The Sidewalk)
Software Dev Assistant	Creativity & Reasoning	AutoGen/CrewAI (The Team)

5.WORKING WITH LANGCHAIN

Working with LangChain

LangChain - The "Swiss Army Knife" of AI

- Instead of a simple layer cake, present **LangChain** as an massive toolkit that allows you to build anything, provided you know which blade to use.
- Without that understanding, it's easy to build something that looks intelligent but isn't reliable.

"LangChain is like a box of Legos. It's easy to start building, but as your castle gets bigger, you realize you need a better foundation to keep it from falling over."

*"Which agent capabilities does **LangChain** provide for me, and which ones must I design myself?"*

Working with LangChain

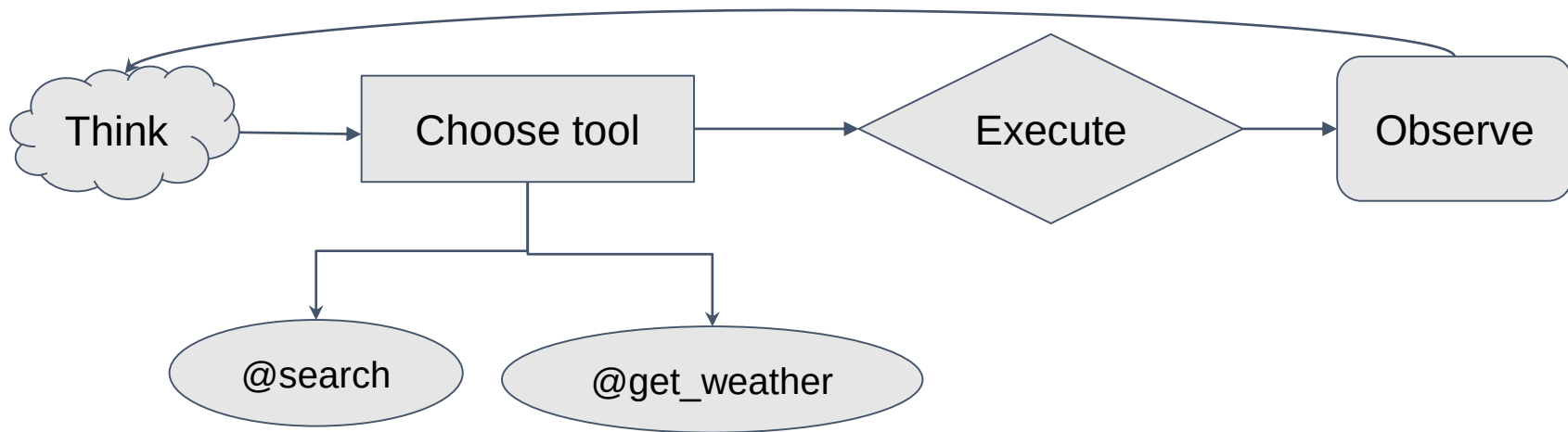
- **The Component Library:** Everything in LangChain is a building block:
 - LLM / ChatModel
 - PromptTemplate
 - Chain
 - Tool
 - Agent
 - Memory
- **The Power of Integration:** It connects your "Brain" (LLM) to "Hands" (Tools) through a simple `create_agent` command.
- **The "Sidewalk" Philosophy:** It creates a straight, easy-to-follow path for standard tasks.

```
from langchain.agents import create_agent
from langchain_openai import ChatOpenAI

model = ChatOpenAI(
    model="gpt-5",
    temperature=0.1,
    max_tokens=1000,
    timeout=30
)
agent = create_agent(model, tools=tools)
```

The "Think-Act-Observe" Loop

Make a repeat



"That's how Langchain got the agent do"

Where is my Mind?

- **The State Crisis**: In **LangChain**, "State" is not a single file. It is scattered everywhere.
 - **Memory Objects**: Only contains the "Partial State" (*what was said in the chat*).
 - **Chain Inputs/Outputs**: Temporary state that vanishes once a step is done.
 - **The Invisible Scratchpad**: Internal reasoning that is **Internal and Invisible** to you as a developer.

External State: Any data in your database or app is handled separately in your **Application Code**.

Let's remind

State in LangChain Is Distributed Across:

=> Conversation / Memory objects

=> Chain inputs & outputs

=> Agent scratchpad (internal reasoning)

=> Your application code (external state)

- State via *Memory* (Partial State).
- State via *Chain Inputs / Outputs*
- State via *Agent Scratchpad* (Internal, Invisible)

Technical Deep Dive

Step 1: Think:

“I need to schedule a follow-up appointment.”

Step 2: Choose Tool:

“Check doctor availability.”

Step 3: Execute:

Call calendar API.

Step 4: Observe:

“No slots available in the morning.”

 **Repeat means:**

Think again:

“Try afternoon slots.”

Define tools

```
@tool
def check_doctor_availability(
    date: str,
    time_of_day: Literal["morning",
"afternoon"]
) -> str:
    slots = _initdb_get(date,
{}).get(time_of_day, [])
    if not slots:
        return f"No slots available in the
{time_of_day} on {date}."
    return f"Available {time_of_day} slots on
{date}: {', '.join(slots)}"

@tool
def book_appointment(date: str, time: str) -
> str:
    #call API here
    return f"Booked follow-up appointment
on {date} at {time}."
```

How LangChain automates reasoning and tool usage

- **Step 1: The Reasoning Engine (@tool definition):** Define specialized tools like `check_doctor_availability` and `book_appointment` using decorators to make them accessible to the agent.
- **Step 2: The Logic Cycle (Code Implementation):** Use `create_tool_calling_agent` to bind the LLM, a list of tools, and a structured prompt together.
- **Step 3: The Execution Loop:** The `AgentExecutor` manages the repetition:
 - **Think:** "I need to check availability for the morning."
 - **Act:** Calls the calendar API via the defined tool.
 - **Observe:** "No slots available in the morning."
 - **Repeat:** "Think again: Try afternoon slots."

Step 1:

```
TOOL check_doctor_availability(date, period,  
doctor_id)
```

```
CALL calendar system # Check lịch khám bác sĩ  
RETURN available_slots OR "no slots"
```

```
TOOL book_appointment(date, time, doctor_id,  
patient)
```

```
CALL booking system # Check xong bắt đầu đặt lịch  
RETURN confirmation
```

Step 2:

INITIALIZE LLM

PROMPT =

You are an appointment scheduling agent.

Goal: Book an appointment for the patient.

Rules:

- Always check availability before booking
- If no slots, try another time period
- Stop when booking succeeds

AGENT = bind(
 llm = LLM,

tools = [check_doctor_availability,
 book_appointment],

prompt = PROMPT

)

Step 3:

INPUT: user_request

STATE:

goal = "book appointment"

tried_periods = []

appointment_booked = false

What “Repeat” Really Means

REPEAT ≠ loop blindly

REPEAT =

Update internal state

Adapt strategy based on
observation

Try a different option

Example

Morning → no slots

↓

Update state: morning_failed = true

↓

Afternoon → try again

LangChain MAF View

1. State

- AgentExecutor / LangGraph maintain **execution state**
- Tracks current step, tool results, loop iteration

2. Memory

- **Short-term:** Conversation memory for context.
- **Long-term:** Integration with Vector Stores (FAISS, Chroma) for RAG capabilities.

3. Planning

- Detect follow-up requirement
- Check availability
- Book slot

4. Orchestration

- Tool abstraction (@tool)
- Multi-tool and multi-agent coordination

5. Streaming

UI shows:

- “Checking availability...”
- “No morning slots”
- “Trying afternoon...”

6. Human-in-the-Loop

- Nurse confirms or adjusts appointment
- Human handles edge cases

6. USE CASES

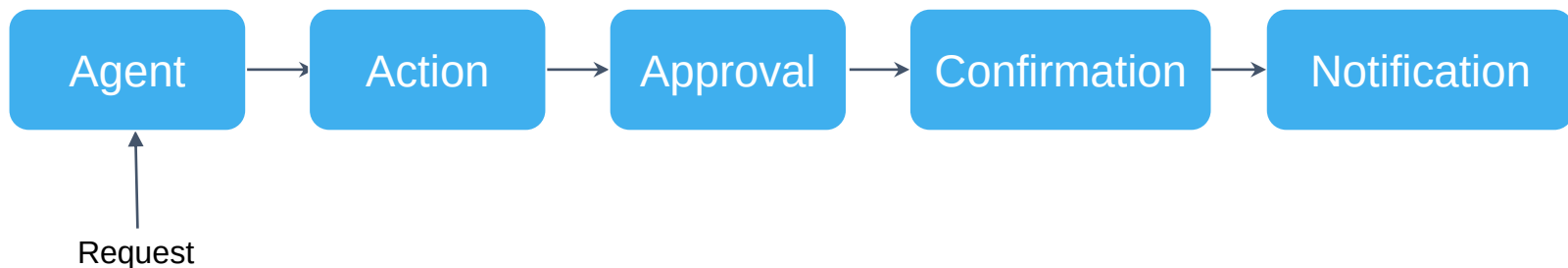
END-TO-END PROJECT

USE CASES

END-TO-END PROJECT

This project ***appears throughout*** your training sessions: "***Hospital Follow-Up Appointment Scheduling Agent***".

Basic flow:



Brainstorm questions:

How capabilities applied in agent? Keep ***state*** for the scheduling process,...?

Which patterns picking for this project? Simple or complex?

When the user or system sends a scheduling request, the agent activates with all the capabilities we studied earlier:

State: Where am I in the scheduling process?

Memory: What patient preferences have been collected?

Planning: What should I do next?

Orchestration: What tool or API do I call?

Streaming: What should I show to the user?

Human-in-the-loop: When do I pause and wait for approval?

‘Agent received request... retrieving patient info... checking doctor availability...’  **streaming** — giving the user visibility.

Brainstorm for your project

In **Action** time, how can we choose design pattern? - ***Let make requirements.***

Approval steps is where the MAF Landscape relevance, watching trade-offs with some MAF

Capabilities → what the agent can do	State, memory, planning, HITL, orchestration.
Design Patterns → how the agent is structured	Single-agent vs supervisor vs workflow/DAG.
Landscape → where the framework fits	Chain vs workflow vs multi-agent.
Trade-offs → why we choose one approach over another	Safety, speed, control, flexibility.

Project suggestion: ***Smart Onboarding Agent, Personal Finance Advisor Agent,...***
Demo source code: [click here](#)

“Every MAF project follows the same structure, only the problem changes.”

You can follow the Langchain framework:

- **Think → Choose Tool → Execute → Observe → Repeat**

OR, expertise in other MAF with this checklist:

- What is the **goal**? ☒
- What does the agent need to **keep track of**? ☒
- Which memories? Past decisions? Failed attempts? ☒
- What are the major steps? ☒
- **Must lists**: APIs, Databases, External services ☒
- Which tool depends on output of another? ☐
- When does the agent **retry**? ☐
- Last but not least, users must see what the agent is doing. *Streaming* is about **transparency**, not speed. ☐





7. OPEN Q&A

Thank you for being a part of this presentation!